

藍圖與建造者：為 AI 開發裝上方向盤與煞車

以 SDD 與 TDD 駕馭 AI 的開發新範式

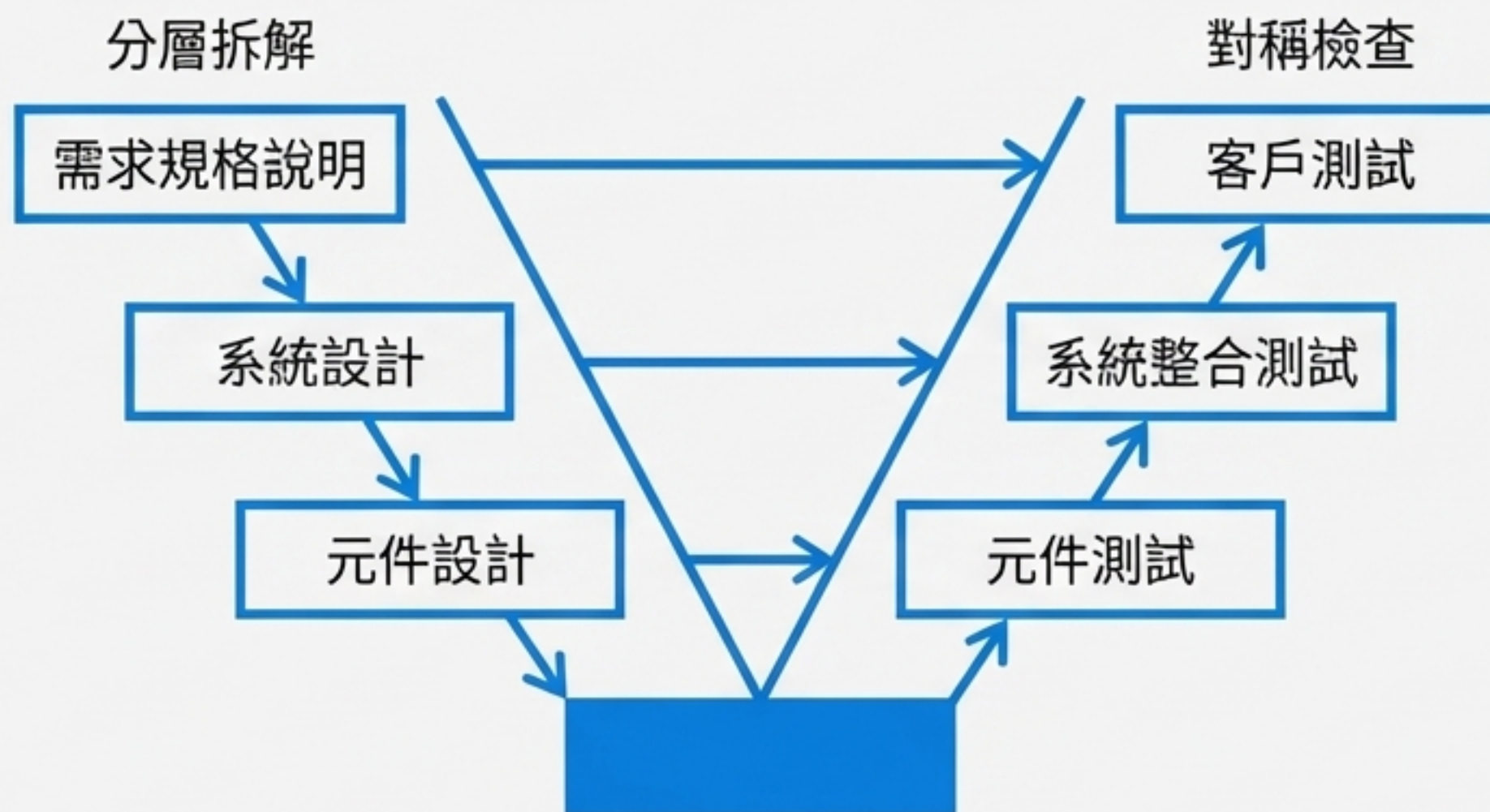


藍圖

AI

傳統的基石：V-MODEL 計畫導向開發

V-model 是一種計畫導向 (Plan-driven) 的開發流程，它將開發活動與驗證活動一一對應，如同對稱的拼裝說明書。



僵化的圖紙，趕不上變化的世界



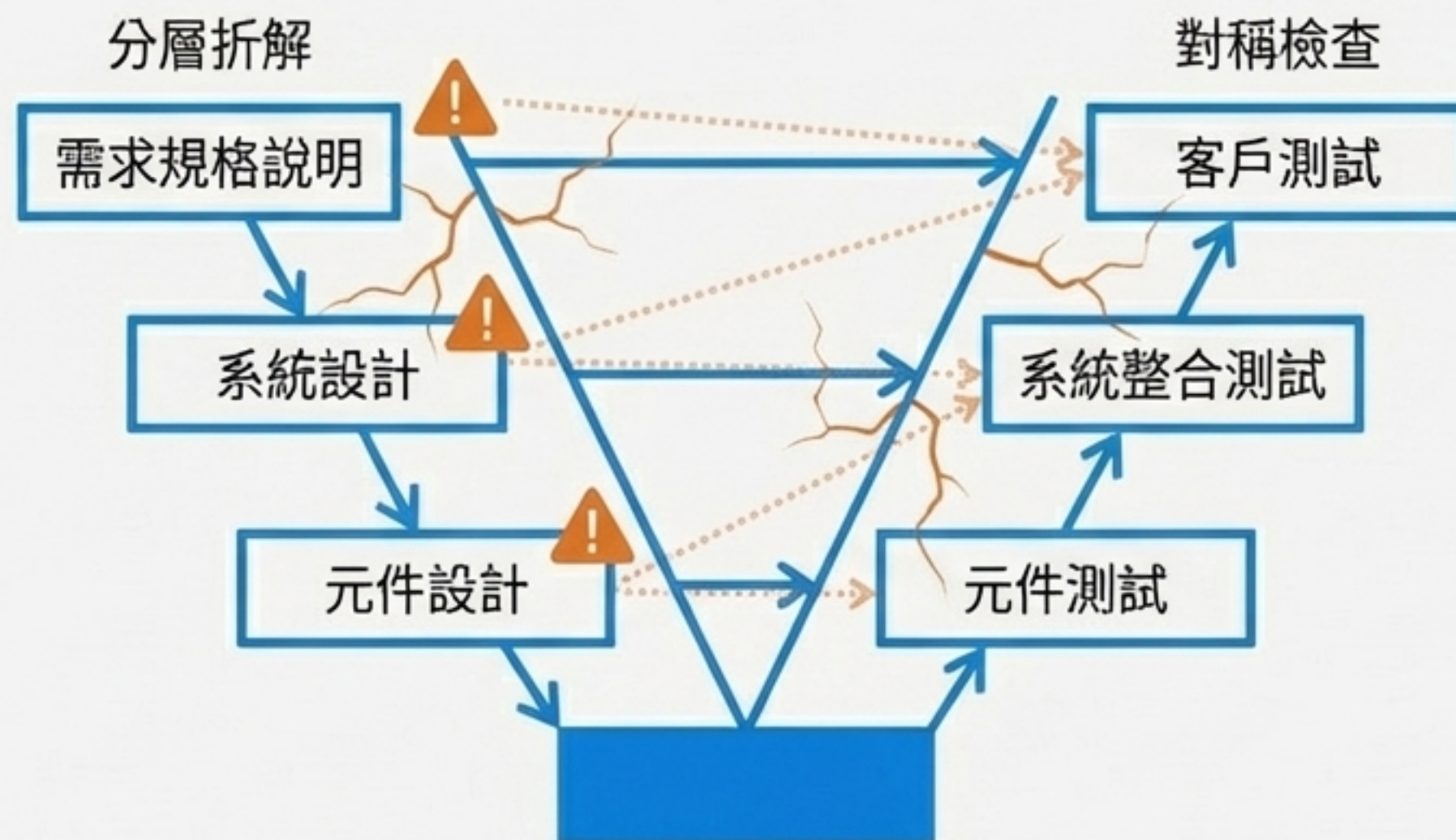
回饋延遲：當你在拼屋頂時，才發現地基的設計算錯了。



需求過早凍結：說明書不能改，最終蓋出不符真實需求的系統。



昂貴的重工：任何需求的變動，都意味著極高的重工成本（Rework）。



智慧拼裝助手：AI 如何加速開發



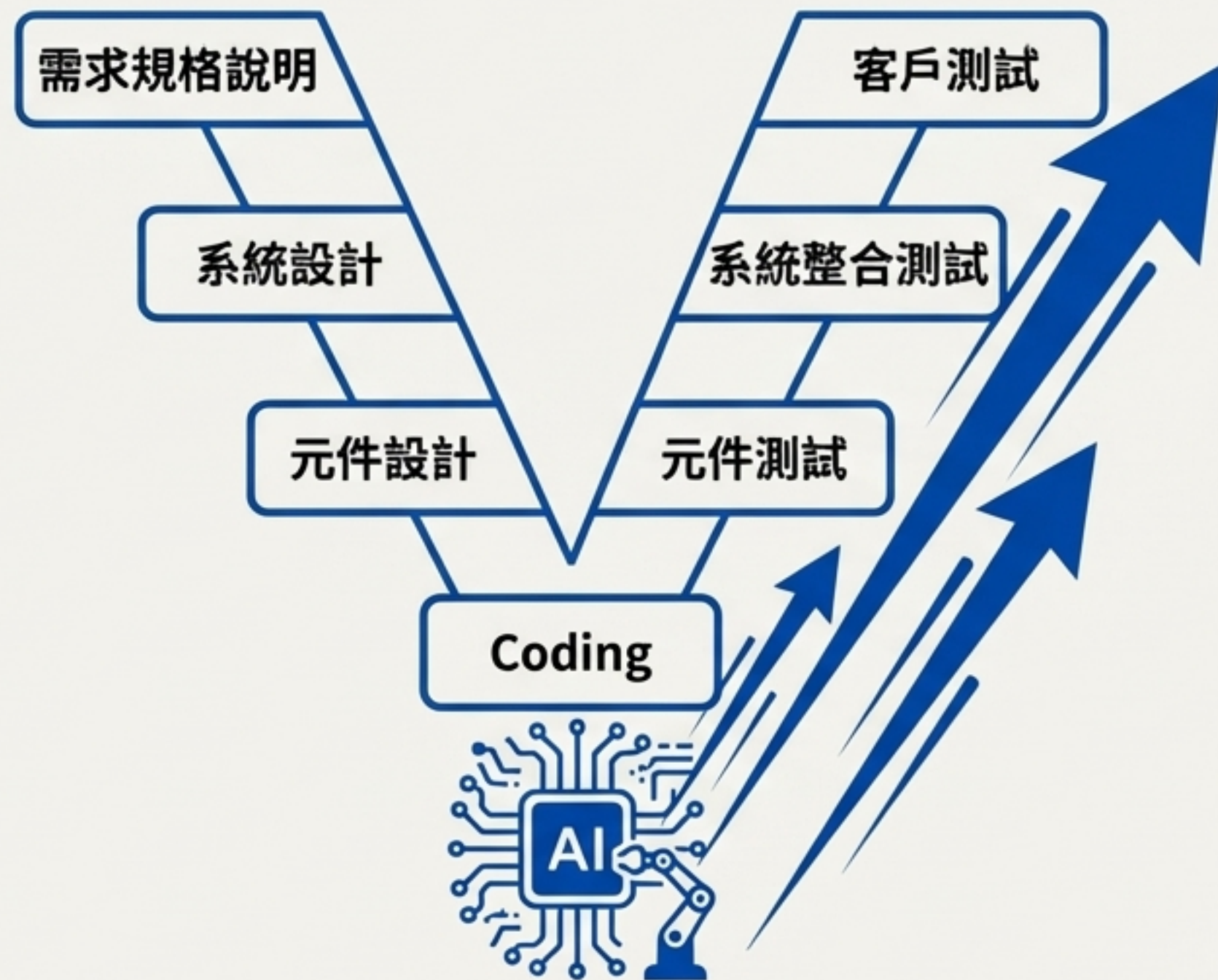
自動化生成：從圖紙（模型）直接生成積木（程式碼）。



軟體分析：像經驗豐富的建築師一樣預測風險，協助決策。



快速原型：讓客戶能先看到城市雛形，降低重工成本。



助手也可能出錯：AI 輔助開發的新挑戰



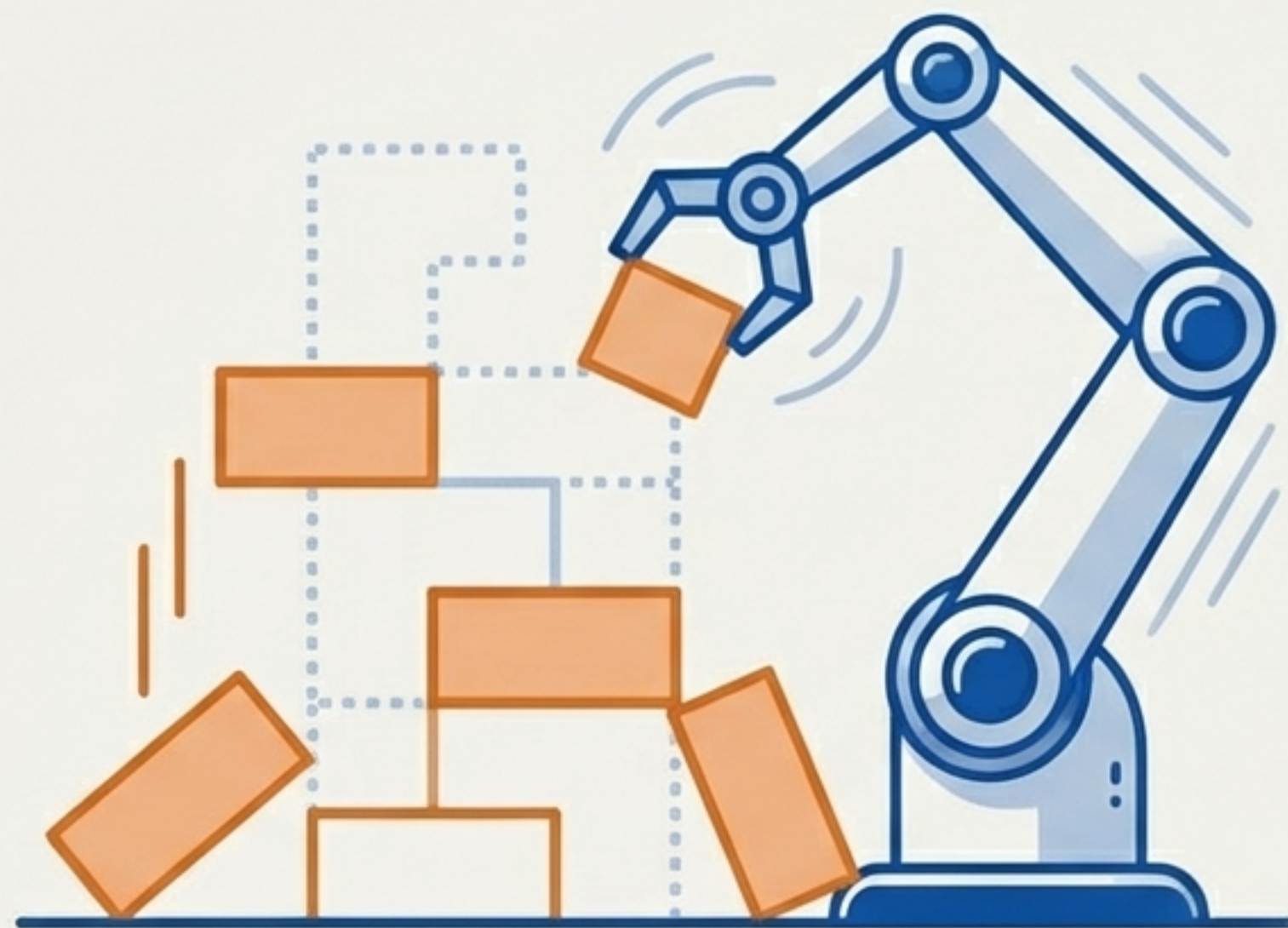
可信度與不確定性：產生虛假警報（False Positives）或存在缺陷的設計。



缺乏脈絡理解：不懂人類的法律、規則或社會技術因素。

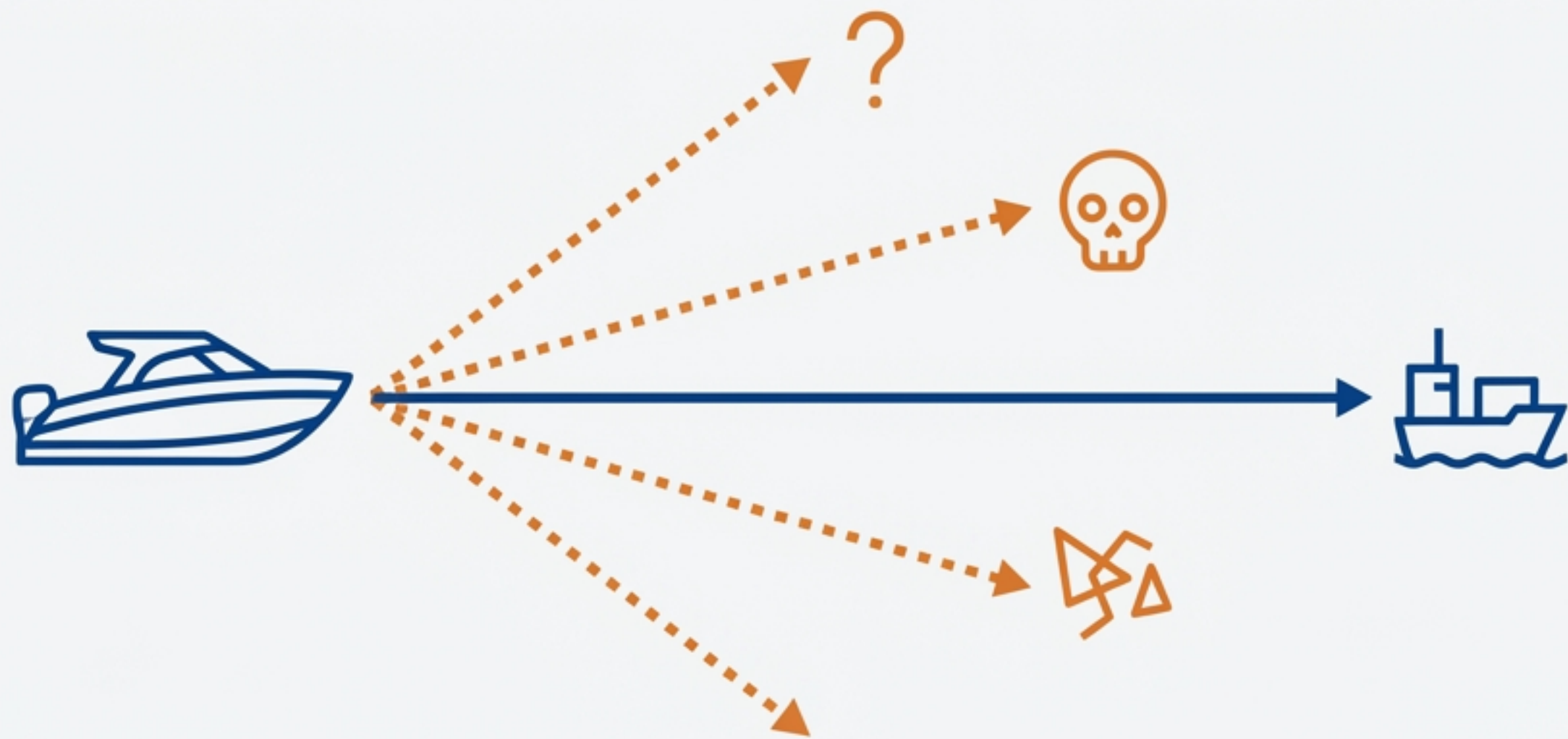


黑盒子的風險：使用外部 AI 服務，一旦出問題，很難進行深入修復。



核心挑戰：「幻覺」與產出的發散

AI 就像一艘超高速快艇，如果你給的導航指令是模糊的，它可能會在幾分鐘內幫你蓋出一座你根本不需要的建築。



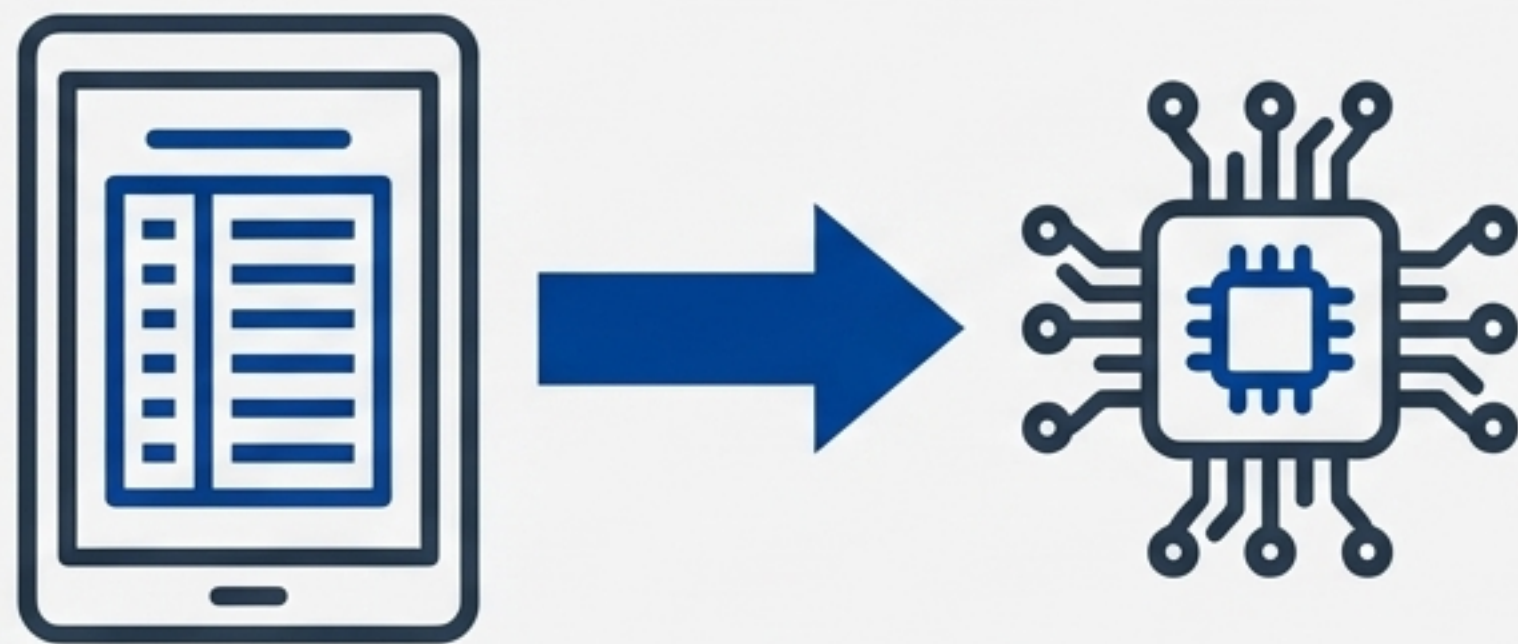
第一道護欄：SDD，大腦中的精準藍圖

What is SDD?

規格驅動開發 (SDD) 強調在動手前，必須先有清晰、完整且可執行的規格說明 (SPEC)。

Why SDD?

它是確保 AI 這艘超高速快艇，能航向正確港口的羅盤。



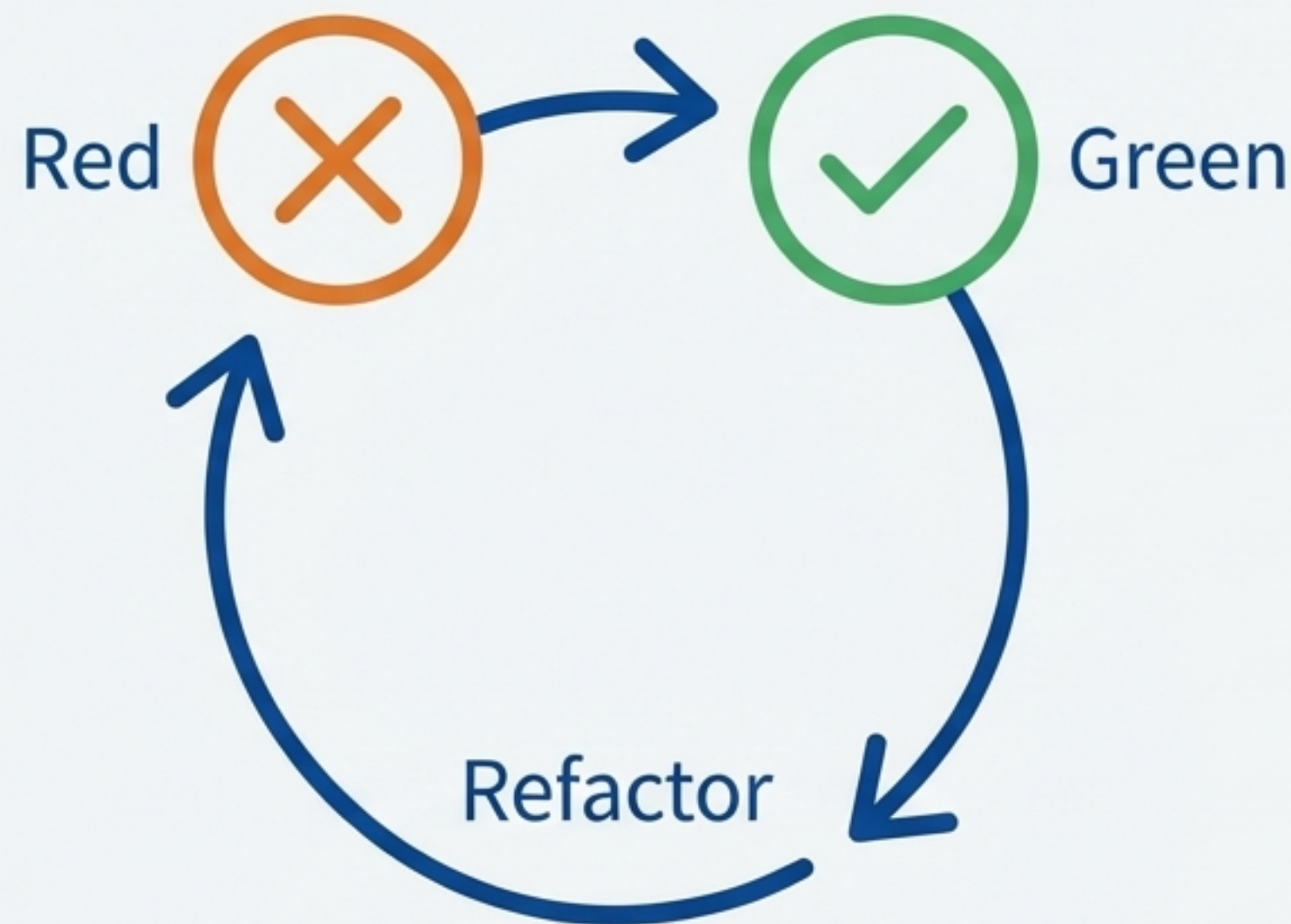
第二道護欄：TDD，積木的品質檢驗官

What is TDD?

測試驅動開發 (TDD) 是「先寫測試，再寫功能」的開發流程 (Red/Green/Refactor)。

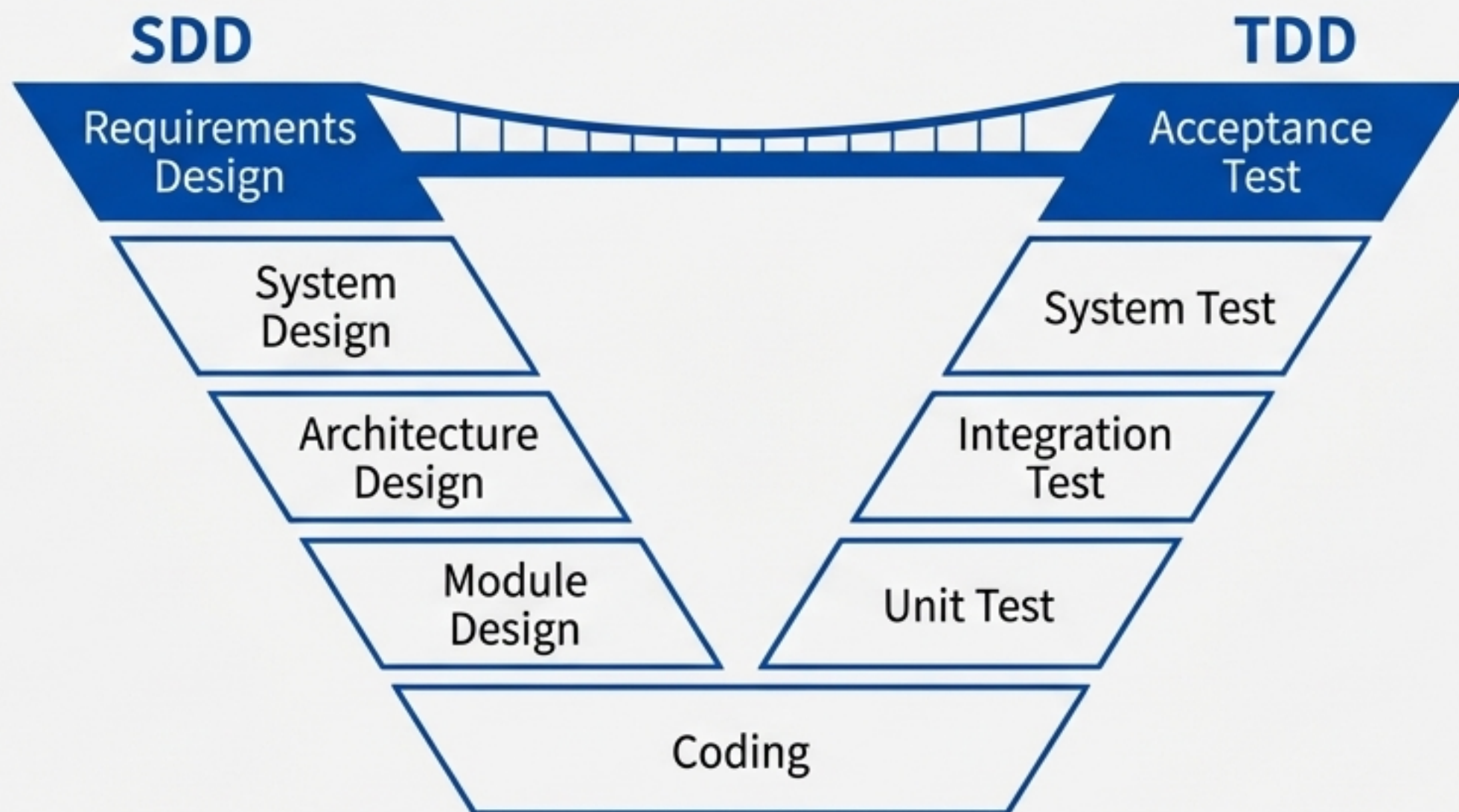
Why TDD?

面對 AI 產出的程式碼，讓 AI 的代碼去「挑戰」你寫好的測試，是驗收成果最有效率的方式。

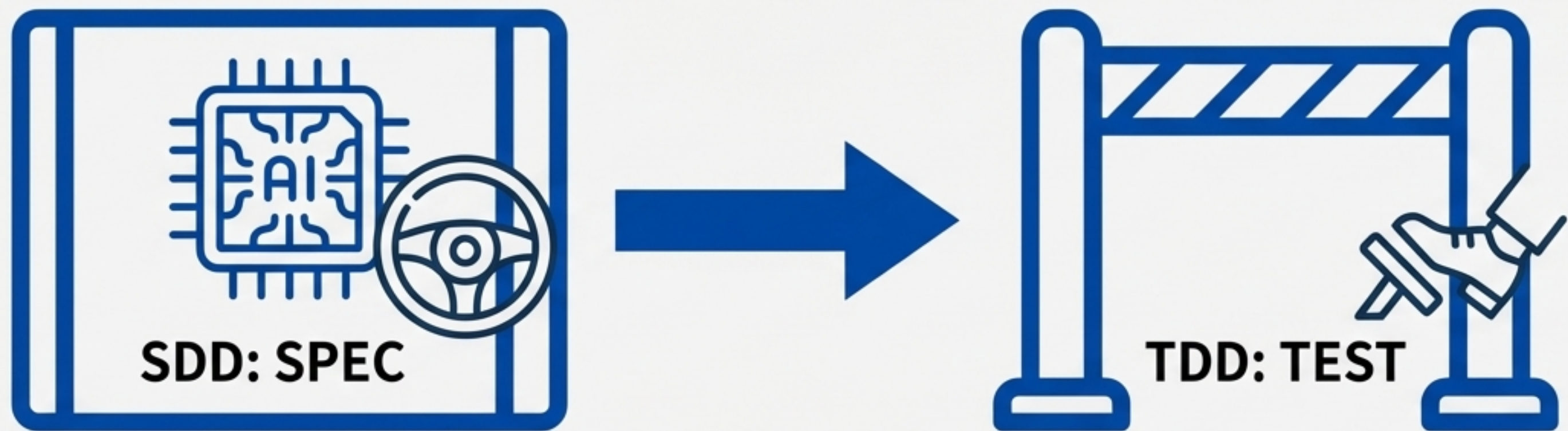


跨越山谷的吊橋：SDD 與 TDD 的新角色

SDD 是 V-model 左側的「設計錨點」，TDD 是右側的「驗證錨點」。AI 讓谷底的編碼變得極快，SDD 與 TDD 則是在山谷兩端架起的吊橋，建立直接的連繫。



給 AI 裝上方向盤與煞車

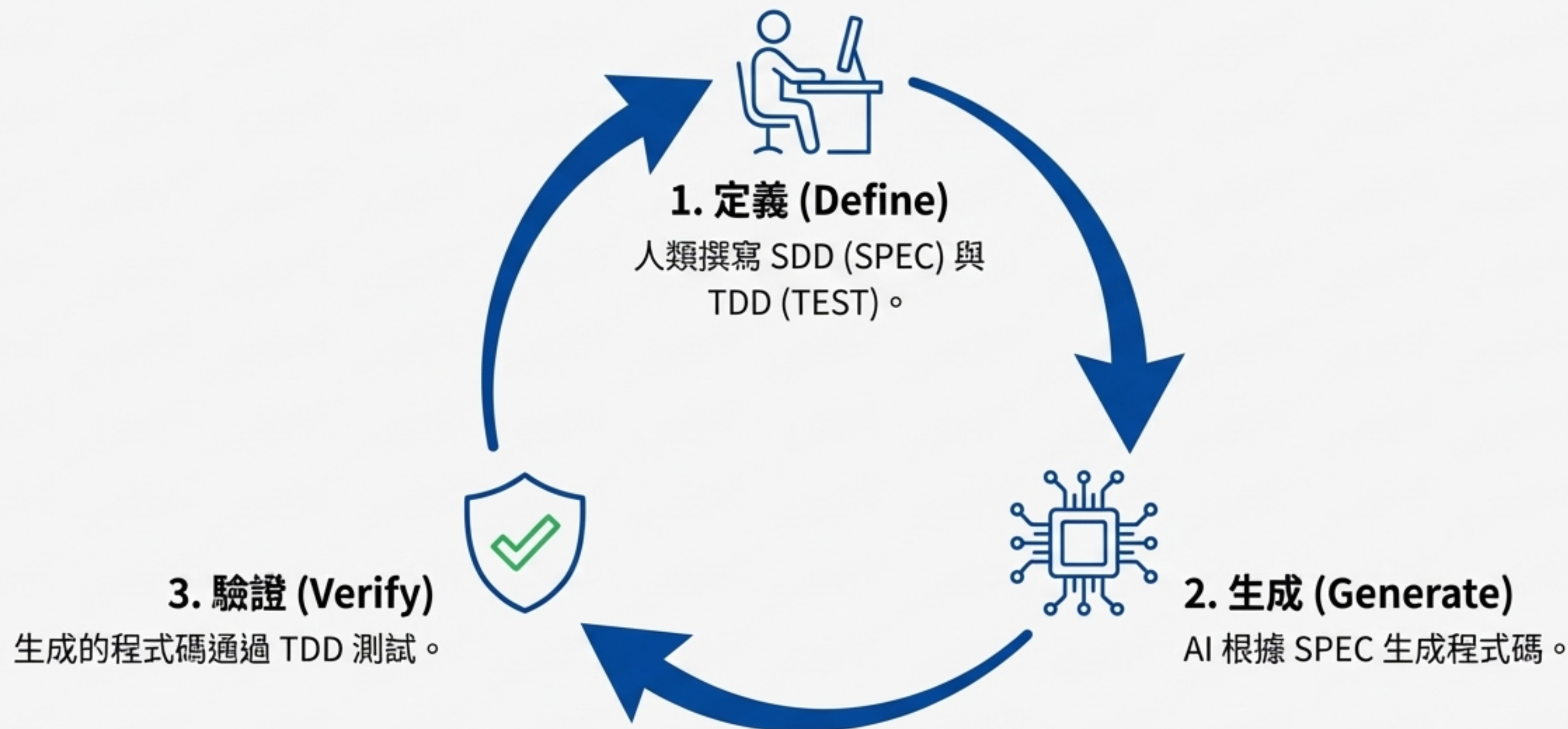


框定路線 (方向盤)：透過 SDD 提供明確的「Why」與「What」，為 AI 框好了行進路線。



自動化驗收 (煞車)：讓 AI 產出的代碼去跑 TDD 的測試案例，測試通過即代表驗收完成。

定義 → 生成 → 驗證：AI 時代的新循環



更精準、更快速、更經濟



減少人工審查

自動化驗收，大幅減少
Code Review 負擔。



收斂 AI 產出

精準的目標，讓 AI 的產
出不再發散。



降低資源浪費

避免因反覆修改而浪費
昂貴的人力與 Token。

這不是萬靈丹：需考量的成本與適用性



- 規格的挑戰：撰寫與維護一份好的規格，本身就是一項專業工作。
- 成本與負擔：雙重文件化（規格+測試）對於需求快速變化的專案，可能成本過高。

從拼裝說明書到數位藍圖：重新定義人與 AI 的協作

AI 是史上最強大的「建造者」。我們的任務，是成為最卓越的「建築師」，為它提供清晰的藍圖 (SDD) 與嚴謹的品管 (TDD)。

